



TRABALHO
Linguagens de Programação
Prof. Lucas Ismaily

SOBRE ORIENTAÇÃO A OBJETOS [10 pontos]

Desenvolvimento de um Sistema de Gerenciamento de Tarefas Pessoais

Descrição Geral:

O trabalho consiste no desenvolvimento de um sistema informatizado para auxiliar usuários na organização de tarefas diárias e no acompanhamento de suas atividades pendentes, concluídas e atrasadas.

O sistema deverá permitir que o usuário registre tarefas, organize suas prioridades, defina prazos de execução e acompanhe o progresso de suas atividades ao longo do tempo.

A aplicação deverá possuir uma interface simples e funcional, permitindo ao usuário cadastrar tarefas, atualizar o status das atividades, visualizar tarefas pendentes e consultar relatórios de produtividade.

Objetivo do Trabalho

Desenvolver uma aplicação completa que auxilie no gerenciamento de tarefas pessoais, aplicando conceitos de:

- Programação orientada a objetos (não necessariamente utilizando todos os conceitos)
- Persistência de dados
- Interface de usuário
- Regras de negócio
- Geração de relatórios

Os alunos deverão demonstrar capacidade de analisar requisitos, modelar classes, implementar funcionalidades essenciais e apresentar a solução de forma organizada.



Requisitos mínimos do sistema

1. Cadastro de tarefas

O sistema deve permitir que o usuário cadastre novas tarefas informando:

- título da tarefa
- descrição
- prioridade (baixa, média ou alta)
- data limite

2. Listagem de tarefas

O usuário deverá poder visualizar uma lista com todas as tarefas cadastradas, indicando:

- tarefas pendentes
- tarefas concluídas
- tarefas atrasadas

3. Atualização de status

O sistema deve permitir alterar o status de uma tarefa para:

- pendente
- em andamento
- concluída

4. Remoção de tarefas

O usuário poderá remover tarefas que não sejam mais necessárias.

5. Relatórios de produtividade

O sistema deve apresentar relatórios simples contendo:

- número de tarefas concluídas
- número de tarefas pendentes
- quantidade de tarefas concluídas por dia

6. Cadastro de usuários

O sistema deverá permitir que diferentes usuários possuam contas separadas, mantendo seus dados e tarefas individualmente.

7. Efetuar login

O sistema deverá permitir que o usuário efetue login de maneira minimamente segura (no mínimo



guardar a senha criptografada)

Requisitos adicionais (opcionais)

- Sistema de lembretes para tarefas próximas do prazo
- Exportação de tarefas em PDF
- Interface web, desktop ou mobile responsiva
- Ordenação de tarefas por prioridade ou data



SOBRE LINGUAGEM FUNCIONAL [10 pontos]

Você pode utilizar **Python em estilo funcional** ou qualquer outra linguagem funcional.

Importante:

- É proibido utilizar **laços de repetição** (for, while).
- Não é permitido utilizar **variáveis auxiliares para armazenamento futuro**.
- O código deve utilizar **funções puras sempre que possível**.
- O uso de **if/else é permitido**.

A entrada de todas as questões deve ocorrer pela **entrada padrão (teclado)**. A saída deve imprimir **apenas o resultado**, sem mensagens adicionais.

Questões 1. Faça um programa que recebe uma lista de números inteiros e imprime a soma de todos os números pares da lista.

Questões 2. Faça um programa que recebe uma lista de números inteiros e imprime somente os números que estão repetidos na lista.

Questões 3. Faça um programa que recebe uma lista de números inteiros e retorna uma nova lista contendo apenas os números ímpares que aparecem pelo menos duas vezes na lista.

Questões 4. Faça um programa que recebe uma lista numérica e imprime uma tupla contendo a média aritmética e a média harmônica dos elementos da lista.

Questões 5. Implemente um programa que receba um número natural n e calcule a soma de todos os números naturais de 1 até n .



SOBRE PROGRAMAÇÃO LÓGICA [10 pontos]

Utilizando **Prolog**, resolva os seguintes problemas.

Questões 1. Implemente um **grafo simples não orientado** representando conexões entre cidades.

Utilize os nomes cidade1, cidade2 etc.

O sistema deve permitir consultas como:

- verificar se duas cidades são diretamente conectadas

?- conectado(cidade1 , cidade2).

- verificar a cidade que tem a maior vizinhança (X é a cidade conectada com mais cidades)

?- maior_vizinhança(Cidades, X).

- verificar se existe alguma ilha, isto é, uma cidade que não tem nenhuma vizinhança (X é uma ilha, se existir).

?- verifica_ilha(Cidades, X).

Questões 2. Implemente os seguintes predicados genéricos sobre listas (sem utilizar o módulo **lists** do SWI-Prolog):

- adiciona_inicio(X, L1, L2) $\rightarrow$ L2 é a lista L1 com o elemento X no início
- remove_elemento(X, L1, L2) $\rightarrow$ L2 recebe a L1 com a remoção da primeira ocorrência de X
- junta_listas(L1, L2, L3) $\rightarrow$ L3 recebe a concatenação das duas listas L1 e L2
- pertence(X, L) $\rightarrow$ verifica se um elemento X pertence à lista L
- tamanho(N, L) $\rightarrow$ N é o número de elementos da lista L

Questões 3. Implemente um programa em Prolog sobre a seguinte família fictícia:

A família Oliveira possui os seguintes registros:

- Roberto e Carla são pais de Lucas e Marina
- Lucas é pai de Pedro
- Marina é mãe de Sofia
- Pedro e Sofia tiveram um filho chamado Daniel



a) Utilizando o predicado $\text{progenitor}(X, Y)$ represente todas as relações de progenitor da família.

b) Implemente predicados para representar as seguintes relações:

- $\text{masculino}(X) \rightarrow$ verdadeiro se X for masculino, falso caso contrário
- $\text{feminino}(X) \rightarrow$ verdadeiro se X for feminino, falso caso contrário
- $\text{irmao}(X, Y) \rightarrow$ X é irmão de Y
- $\text{irma}(X, Y) \rightarrow$ X é irmã de Y
- $\text{pai}(X, Y) \rightarrow$ X é pai de Y
- $\text{mae}(X, Y) \rightarrow$ X é mãe de Y
- $\text{avou}(X, Y) \rightarrow$ X é avô de Y
- $\text{avo}(X, Y) \rightarrow$ X é avó de Y
- $\text{tio}(X, Y) \rightarrow$ X é tio de Y
- $\text{tia}(X, Y) \rightarrow$ X é tia de Y
- $\text{primo}(X, Y) \rightarrow$ X é primo de Y
- $\text{prima}(X, Y) \rightarrow$ X é prima de Y
- $\text{descendente}(X, Y) \rightarrow$ X descende de Y



INFORMAÇÕES IMPORTANTES

A nota do trabalho será a média aritmética das notas nos três paradigmas: Orientação a Objetos, Funcional e Lógico.

A entrega será **somente** por e-mail: ismailybf@ufc.br, numa pasta zipada com todas as questões (no caso da questão de Orientação a Objetos, você pode zipar o projeto) e os nomes dos membros juntamente com as matrículas. **Importante:** cada questão de funcional e lógico deve estar em um arquivo separado. O prazo máximo da entrega do trabalho é para o dia **29/06/2026**.

As partes funcional e lógica serão corrigidas de forma automatizada, então preste bastante atenção. Sobre as questões de funcional: a entrada de todas as questões é pelo teclado (entrada padrão). A única coisa impressa na tela deve ser o resultado. Qualquer impressão diferente do resultado, como por exemplo, “digite a entrada”, “a saída é” será considerado resposta errada. Bem como será considerado errado se for impresso “[1,2,3]” no lugar de “1 2 3”. Se a entrada for uma lista numérica, será digitado apenas os números, por exemplo, “1 2 3” no lugar de “[1,2,3]”. Sobre as questões de prolog: implemente os predicados utilizando exatamente os mesmos nomes e número de argumentos descritos nas questões. Você pode ter predicados auxiliares, porém não pode mudar o nome e nem a quantidade de argumentos.

Trabalho em grupo com no máximo 5 membros e no mínimo 3. Sejam honestos com vocês e comigo, por favor. Se for detectado qualquer tipo de fraude, os envolvidos receberão nota zero. **Note: os envolvidos receberão zero, não importa se a equipe foi a origem ou o destino, ambas receberão nota zero.**